Stochastic Anomaly Simulator

Project Documentation

Pole Anomaly Project

May 10, 2026

Contents

1 Introduction

This document includes a general overview of the Pole Anomaly Project. The overall objective of this project is anomaly detection and classification using Neural Networks.

A critical component of this machine learning pipeline is the generation of training data. The data generation module is designed to inject, user-defined anomalies (such as Squares, Exponential Curves, and Bell Curves) into synthetic radiation noise based on real-world sensor baselines. The synthetic datasets generated from these anomaly templates will be directly used to train, validate, and test the neural networks for anomaly detection.

2 Quick Setup Guide

This project isolates dependencies using a Python virtual environment to ensure absolute reproducibility.

2.1 Environment Setup

To initialize the project on a new machine, execute the following commands in the root of the project:

```
# 1. Create a Python Virtual Environment
py -3.13 -m venv .venv

# 2. Activate the Environment (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# 3. Install strictly required numerical libraries
pip install -r requirements.txt
```

2.2 Dependencies

The `requirements.txt` installs the following core standard math libraries:

- `numpy` & `pandas`: For fast mathematical array manipulations and template CSV handling.
- `plotly`: For high-fidelity, interactive visualizations saved in HTML logs.
- `scipy`, `statsmodels`, `tslearn`: Required specific dependencies for statistical interpolation and historical legacy dataset tests.

3 Synthetic Data Generation Pipeline (data-gen)

As the repository expands to include other projects (such as neural networks for anomaly detection), this section specifically details the anomaly generation pipeline.

3.1 File Structure Review

The repository enforces a strict separation of concerns, divided into modular directories:

- data/: Located at the root, contains raw background sensor readings (e.g., 2015_months_DebitDo used as a baseline.
- data-gen/: The main pipeline directory containing the synthetic dataset generator and its components:
  - data-gen/anomalies/: Contains the normalized .csv blueprints (templates) for different anomaly shapes (e.g., exp_square.csv, bell.csv, m_sq.csv).
  - data-gen/data/: The destination for the final generated output datasets (e.g., synthetic_custom_dataset.csv) utilized for neural network training.
  - data-gen/logs/: The destination for Plotly's interactable HTML visualizations and PNG exports. Provides a pure geometric proof of concept without requiring external dashboard readers.
  - data-gen/src/: Stores the pure mathematics and geometric generator libraries, decoupled from direct script logic.
  - data-gen/script/: Stores execution pipelines and visualization launchers. Scripts utilize the math from data-gen/src/ to generate datasets or debug plots.

3.2 Explanation of the Engine and Scripts

3.2.1 data-gen/src/anomaly_generator.py

This is the core physics computational engine. It provides the generate_anomaly() function which transforms a normalized CSV blueprint into a physical discrete array anomaly string. The engine mathematically scales the template temporally (period span) and vertically (amplitude intensity) while embedding Gaussian "deformation" limits declared in the CSV file logic.

3.2.2 data-gen/script/generate_custom_dataset.py

The final execution pipeline for training. It analyzes the original physical gamma sensor data using moving averages to dynamically separate high-frequency point-to-point noise from the slow, low-frequency sensor drift. It then mathematically simulates a highly realistic **wandering baseline** using a chunk-by-chunk Ornstein-Uhlenbeck (OU) mean-reverting random walk process, overlaying the high-frequency Gaussian noise on top.

Spanning datasets of up to 10,000,000 steps, it spontaneously samples anomaly templates from the library, calculates randomized injection amplitudes and duration bounds, and seamlessly injects them into the wandering baseline. It maps each injected shape

to a unique multi-class integer ID, tagging these absolute ground-truth labels alongside cleanly tracking the pure anomaly wave.

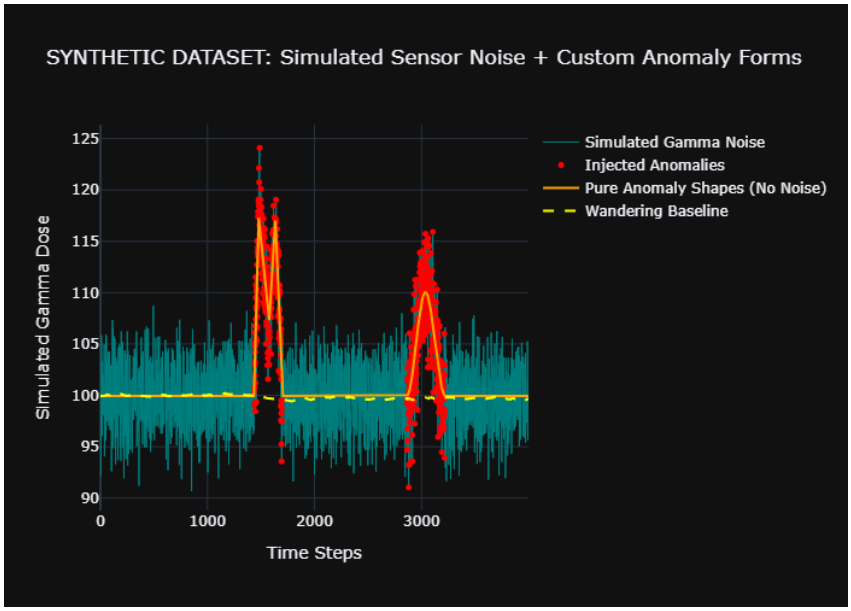

Figure 1: Output of generate_custom_dataset.py

3.2.3 `data-gen/script/test_generator_deformation.py`

A visual sandbox and boundary debugging tool. Iterating automatically over all discovered templates in the `data-gen/anomalies/` directory, it runs exactly 5 localized chaotic generation iterations of each template to visually verify and validate the mathematical variability limits enforced by the template's parameters. Exports directly to `anomaly_template_deformation_test.html` and `.png`.

3.2.4 `data-gen/script/visualize_anomalies_templates.py`

This script simply visualizes the pure original mathematical blueprints. It renders the pure lines and graphs the interpolation properties exactly as described in the blueprint CSV to a localized HTML and PNG, without injecting randomization or temporal noise.

3.3 Anomalies Templates Creation

Instead of relying on rigid pre-recorded dataset distributions like legacy UCR databases, targeted geometric structures are freely drawn and mathematically described in the `data-gen/anomalies/` folder.

3.3.1 Configuration Blueprint (CSV Logic)

A standard anomaly template requires 7 strict column definitions mapping logic coordinates and behavioral constraints:
`time, value, t_minus, t_plus, v_minus, v_plus, interp`

- **time & value**: Standardized physical shape coordinates (normally scaling from 0.0 to 1.0). They map the major mathematical inflection milestones of the curve.

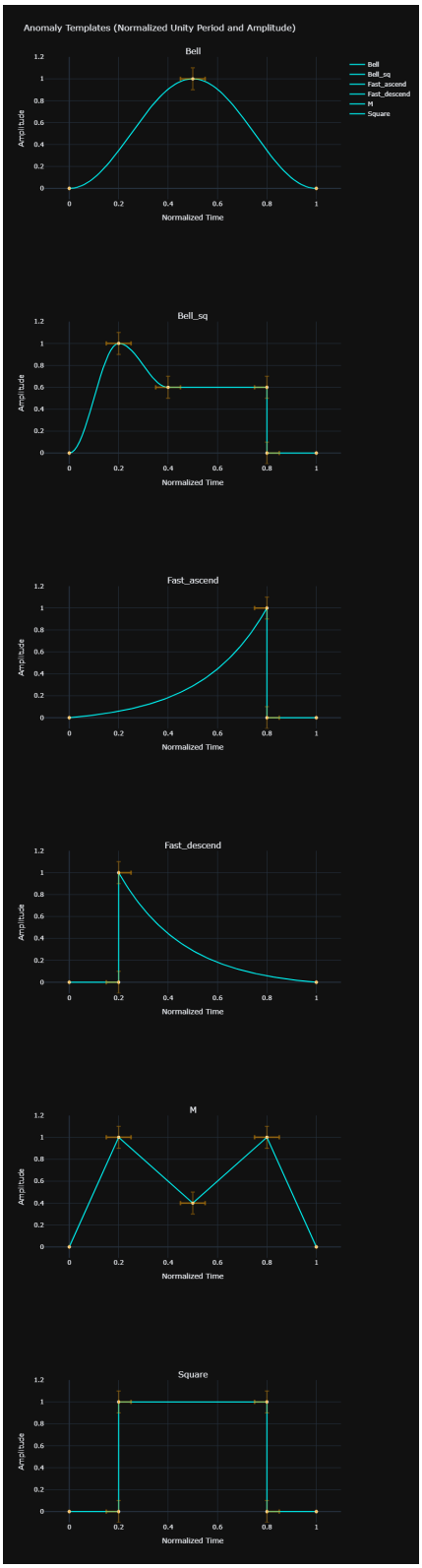

Figure 2: Output of visualize_anomalies_templates.py

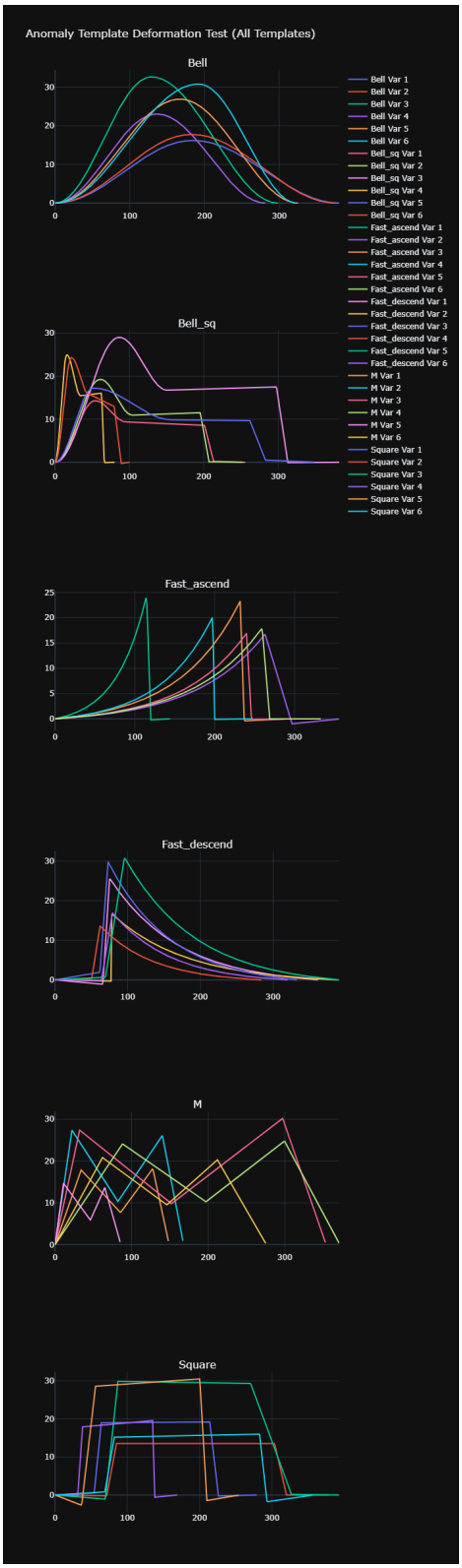

Figure 3: Output of test_generator_deformation.py

- **Deformation Multipliers (t_minus, t_plus, v_minus, v_plus)**: Defines spatial jitter limits per node. A value of `1.0` allows symmetric Gaussian distortion up to 100% of the global script variance parameter. A value of `0.0` creates an unyielding geometric wall perfectly locking the point in that specified space direction.
- **interp (Interpolation Mode)**: Defines the localized mathematical connection trajectory bridging the gap toward the targeted point.
  - `linear`: Connects sequential milestones with a simple Euclidean linear straight interpolation fallback.
  - `exp`: Instigates a smooth sluggish temporal acceleration morphing into a harsh geometric peak, useful for raw exponential growth burst characterizations.
  - `bell`: Executes a fractional phase-shift cosine envelope ($\cos(\pi x)$), generating an intensely organic smooth "S-curve". Ensures a perfect non-rigid mathematical slope shift suitable for defining Gaussian-like topographic bell shapes.

4 Proposed Architectures for Anomaly Identification

Based on the characteristics of the synthetic dataset and the overall project objectives (classification performance, computational cost, and explainability), the following neural network architectures are proposed for anomaly detection and classification.

The simulated data presents specific challenges: extreme high-frequency gamma noise, distinct geometric signatures for anomalies (e.g., Bell, M, Square), and significant temporal and amplitude deformation. The models must be robust to background fluctuations and possess translation and scale invariance to generalize effectively.

4.1 General Data Preprocessing Pipeline

Regardless of the architecture chosen, the continuous sensor data cannot be processed infinitely. The shared preprocessing pipeline involves:

1. **Sliding Window Extraction:** Slicing the continuous gamma dose column into fixed-size overlapping windows (e.g., chunks of 256 or 512 time steps). This converts a single 1D time series of length N into a localized dataset of shape `(Samples, Window_Size, Channels)`.

2. **Labeling Strategy:** For multi-class classification, assigning a categorical integer ID to the whole window corresponding to the specific anomaly shape present (e.g., 1 for Bell, 2 for Square, 0 for normal background). For point-wise segmentation, assigning a class label to every single point in the window to map the exact width and type of the anomaly.

3. **Normalization:** Applying Z-score normalization (subtracting the mean and dividing by the standard deviation) to each window independently. This ensures stability during gradient descent, as the extreme simulated radiation values can easily saturate activation functions like ReLU or Sigmoid.

4.2 1D Convolutional Neural Networks (1D-CNN) / ResNet-1D

1D convolutions are excellent for extracting local morphological features (like the sharp edges of a "Square" or the smooth curve of a "Bell") regardless of where they appear in the time window.

- **Performance & Cost:** Highly efficient and parallelizable, making them computationally cheap to train and deploy for real-time inference on continuous sensor data streams.

- **Explainability:** This architecture is directly compatible with Gradient-weighted Class Activation Mapping (**Grad-CAM**). Heatmaps can be generated over the input time series to visually prove that the model is activating on the structural milestones of the anomaly rather than simply overfitting to background noise.

Detailed Implementation Pipeline

1. **Input Layer:** Accepts a tensor of shape `(Batch_Size, Channels, Window_Size)` (in PyTorch) or `(Batch_Size, Window_Size, Channels)` (in Keras).

2. **Feature Extraction (Convolutional Blocks):** Stack 3 to 5 `Conv1D` layers. Each convolution slides a kernel over the time dimension to detect specific features. Follow each convolution with `BatchNormalization` to stabilize the learning process across batches, a `ReLU` activation to introduce necessary non-linearity, and `MaxPooling1D` to downsample the sequence length. This spatial reduction forces the network to learn higher-level features while increasing the number of filters (e.g., $32 \rightarrow 64 \rightarrow 128$).

3. **Global Pooling:** Use `GlobalAveragePooling1D` (or `AdaptiveAvgPool1d`) to collapse the time dimension entirely. This operation calculates the average output of each feature map across the time sequence, drastically reducing parameters, preventing overfitting, and acting as the structural prerequisite for Grad-CAM visualization.

4. **Output Layer:** A `Dense` (or `Linear`) layer outputting `num_classes` logits, typically paired with a Softmax activation during inference for multi-class probability prediction.

4.3 Convolutional Recurrent Neural Networks (CRNN)

This hybrid approach utilizes a 1D-CNN base to extract local features, followed by a recurrent layer (such as an LSTM or GRU) to process the temporal sequence of those features. This is particularly useful for complex shapes like "M" where the model needs to track a structured sequence of events over time.

- **Performance & Cost:** Yields high accuracy for temporally stretched anomalies. The sequential processing of the RNN component adds a moderate computational overhead compared to a pure CNN.

Detailed Implementation Pipeline

1. **Spatial Feature Extractor:** Pass the input window through a shallow 1D-CNN (e.g., 2 `Conv1D` layers) *without* Global Pooling. The output remains a compressed sequence of dense feature vectors, retaining temporal order but with lower dimensionality.

2. **Temporal Sequence Modeler:** Pass the output sequence directly into a `Bidirectional(LSTM)` or `GRU` layer. The bidirectional nature allows the network to understand context from both the past and the future steps within the window, which is crucial for identifying shapes that have symmetrical properties (like Bells).

3. **Aggregation & Output Layer:** Extract the final hidden state of the RNN (or use an Attention layer over all hidden states) and pass it into a `Dense` layer for final classification.

4.4 Transformers with Multi-Head Self-Attention

Time-series Transformers excel at finding correlations across different parts of the signal. The multi-head self-attention mechanism can learn to suppress the high-frequency gamma noise and focus entirely on the broader structural patterns of the anomalies.

- **Explainability:** Attention mechanisms offer intrinsic explainability. The attention weights can be extracted and visualized to show exactly which time steps the model prioritized when making its classification decision, acting as a powerful and transparent complement to Grad-CAM.

Detailed Implementation Pipeline

1. **Patching/Embedding:** Unlike RNNs or CNNs, Transformers process all inputs simultaneously. Project the input sequence into a higher-dimensional embedding space (e.g., size 64) using a linear layer or a small `Conv1D` to create temporal "tokens".

2. **Positional Encoding:** Add explicit Sine/Cosine positional encodings to the embeddings so the model retains the chronological order of the time steps, since the attention mechanism itself is permutation-invariant.

3. **Transformer Blocks:** Pass the sequence through 2-4 `TransformerEncoder` blocks. Each block contains a Multi-Head Self-Attention mechanism (allowing tokens to interact and weigh each other's importance) and a Feed-Forward Network.

4. **Output Layer:** Aggregate the output via a dedicated CLS token (Class Token) or Global Average Pooling, and classify via a `Linear` layer.

4.5 1D U-Net (Segmentation Approach)

If the objective requires detecting the anomaly exactly at its onset by pinpointing the precise start and end timestamps, framing the problem as a time-series segmentation task is highly effective. A 1D U-Net or Fully Convolutional Network (FCN) outputs a probability score for every single time step, effectively generating a localized bounding mask over the anomaly in real-time.

Detailed Implementation Pipeline

1. **Encoder (Downsampling Pathway):** Use sequential `Conv1D` and `MaxPooling1D` layers to compress the time series. Crucially, save the output tensor at each resolution level before pooling. These are the "skip connections".

2. **Decoder (Upsampling Pathway):** Use `UpSampling1D` (or `Conv1DTranspose`) to gradually expand the sequence back to its original length. At each expansion step, concatenate the corresponding saved skip connection from the Encoder. This mechanism forces the network to combine deep semantic features (from the bottom of the U-Net) with high-resolution spatial context (from the skip connections).

3. **Point-wise Output Layer:** A `Conv1D` layer with a kernel size of 1 and exactly 1 filter per target class, followed by a Sigmoid/Softmax activation. The output shape perfectly matches the original input sequence length.

4. **Loss Function:** Utilize specialized loss functions like **Dice Loss** or **Focal Loss** to handle the severe class imbalance, as the vast majority of points in the dataset represent normal background noise.

5 Reference PyTorch Implementation: 1D-CNN

To validate the proposed architectures, a functional baseline implementation of the 1D-CNN was developed using PyTorch. The architecture has been specifically designed for **Multi-Class Classification**, capable of distinguishing between various anomaly templates (Bell, Square, M, etc.). The codebase is organized into modular scripts within the `1D-CNN` directory, with full CUDA GPU acceleration support.

5.1 Data Processing (`dataset_loader.py`)

The `dataset_loader.py` module bridges the synthetic `.csv` generator and the neural network. It performs the following steps:

1. **Temporal Split:** The continuous signal is split chronologically *before* windowing: the first 80% of time steps are reserved for training and the remaining 20% for testing. This prevents **data leakage**, which would occur with a random split because overlapping windows (stride 32 over a 512-point window) share up to 94% of their data points.

2. **Sliding Window Extraction:** Each split is independently sliced into overlapping windows (configurable size, default 512, stride 32).

3. **Threshold-Based Labeling:** A window is only labeled as a specific anomaly class if the anomaly occupies at least **10%** of the window length. This prevents the CNN from learning that windows containing 99% pure noise should be classified as anomalies. When the threshold is met, the most frequent anomaly class ID within the window is assigned via majority voting (`np.unique` with `return_counts`).

4. **Per-Window Centering (Instance Normalization):** To make the network robust against the slowly wandering background baseline generated by the Ornstein-Uhlenbeck process, each sliding window is independently centered by subtracting its own local mean μ_{window}. However, a global standard deviation σ_{train} is computed across all centered training windows to preserve the relative amplitudes of the spikes. This prevents the model from "squashing" small anomalies into looking like normal noise. The test set is scaled using this same global statistic. Critically, the global standard deviation is persisted to `normalization_stats.json` so that the inference pipeline applies *identical* amplitude scaling:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{window}}}{\sigma_{\text{train}}} \tag{1}$$

5. **Tensor Reshaping:** The output is reshaped into the strict `(Batch_Size, 1, Window_Size)` format required by PyTorch's `Conv1d` layers.

5.2 Architecture Definition (`model.py`)

The `Anomaly1DCNN` class inherits from `torch.nn.Module` and implements a 4-block convolutional architecture with a progressively narrowing kernel strategy to balance global structure capture with fine-grained feature extraction:

- **Block 1:** `Conv1d(in=1, out=16, kernel=7, padding=3)` → `BatchNorm1d` → `ReLU` → `MaxPool1d(2)`.
- **Block 2:** `Conv1d(in=16, out=32, kernel=7, padding=3)` → `BatchNorm1d` → `ReLU` → `MaxPool1d(2)`.
- **Block 3:** `Conv1d(in=32, out=64, kernel=5, padding=2)` → `BatchNorm1d` → `ReLU` → `MaxPool1d(2)`.
- **Block 4:** `Conv1d(in=64, out=64, kernel=3, padding=1)` → `BatchNorm1d` → `ReLU` → `MaxPool1d(2)`.

The early layers use larger kernels (7, 7) to capture broad temporal structure such as the overall envelope of anomaly shapes, while later layers use smaller kernels (5, 3, 3) to refine local features. To prevent catastrophic spatial loss while retaining deep abstraction, the network utilizes a Regional Pooling layer (`AdaptiveAvgPool1d(4)`) instead of Global Average Pooling. This preserves 4 distinct temporal bins, allowing the network to recognize the temporal topography of shapes (e.g., distinguishing the two separate peaks of an `M` shape from a solid `m_sq` block).

This flattened sequence vector of size 256 (64×4) is then passed through a classifier head composed of `Flatten()` → `Dropout(0.4)` → `Linear(256, 32)` → `ReLU` → `Dropout(0.2)` → `Linear(32, num_classes)`. The dropout layers serve as regularization to prevent overfitting. The total model footprint totals approximately 35,141 trainable parameters.

5.3 Training Loop (`train.py`)

The training script establishes the complete PyTorch training ecosystem with GPU acceleration and comprehensive logging:

- **Data Split:** Since the dataset loader already performs the temporal train/test split, the training script further divides the training set 80/20 into train/validation subsets using a random permutation for early stopping monitoring.
- **Class Imbalance Handling:** Uses `np.bincount` to dynamically calculate the distribution of each class. It computes inverse frequency weights as $w_c = \frac{N_{\text{total}}}{C \cdot N_c}$ and passes them to the `CrossEntropyLoss` criterion, forcing the optimizer to treat rare anomaly shapes with the same importance as the dominant normal background.
- **Optimization:** Employs the Adam optimizer with an initial learning rate of 10^{-3}.
- **Learning Rate Scheduling:** A `ReduceLROnPlateau` scheduler (patience 4 epochs, factor 0.5) monitors the validation loss and halves the learning rate when progress stalls. This reduces the oscillatory behavior observed in the validation loss during later training stages.
- **Early Stopping:** A patience-based mechanism (10 epochs) monitors the validation loss. The best model weights are saved to `best_1d_cnn_pytorch.pth` only when the validation loss improves.
- **Evaluation:** After training, the best checkpoint is reloaded and evaluated on the held-out test set. A per-class accuracy breakdown is computed and logged.

- **Logging:** A `.txt` log file is generated in the `logs/` directory, recording all hyperparameters, environment details (PyTorch version, GPU model, CUDA version), class distributions, per-epoch loss curves with current learning rate, training duration, and per-class test accuracy.

5.4 Inference and Evaluation (`visualize_inference.py`)

To interpret the model's behavior on streaming data, a visualization and evaluation script runs the trained model over a continuous segment of the dataset using a fine-grained sliding window (stride of 10). Inference is performed on GPU when available. Two key design decisions distinguish this inference pipeline from naive approaches:

1. **Consistent Normalization:** The inference script loads the global σ_{train} value saved during training from `normalization_stats.json`. It applies the exact same per-window centering strategy by dynamically calculating μ_{window} during the sliding pass. This ensures the model is perfectly invariant to vertical baseline drift while seeing the same scale distribution it was trained on.

2. **Overlapping Window Accumulation:** Instead of assigning each window's prediction to a single center point, the softmax probability vector is **accumulated** across all points covered by the window. With stride 10 and window size 512, each point is covered by up to 51 overlapping windows. The final per-point class probability is the average across all contributing windows:

$$P_{\text{final}}(c \mid t) = \frac{1}{|\mathcal{W}_t|} \sum_{w \in \mathcal{W}_t} P_w(c) \tag{2}$$

   where $\mathcal{W}_t$ is the set of all windows covering time step t. This ensemble-like averaging significantly reduces prediction noise.

The script produces the following outputs:

1. **Interactive Visualization:** An interactive dual-axis Plotly graph rendering the simulated gamma noise on the primary axis and distinct, colored probability curves for each anomaly class on the secondary axis. A rolling average (window of 30) is applied to smooth the accumulated probabilities.

2. **Event-Level Evaluation:** Rather than evaluating per-point predictions—which are misleading due to sliding window boundary spillover inflating false positives—the evaluation groups contiguous anomaly points into discrete **events**. For each event, the model's accumulated probability vectors are averaged and the `argmax` class is compared against the ground truth label. The log reports a per-event results table, an event-level confusion matrix, and derives per-class Precision, Recall, and F1-Score at the event granularity, along with the overall detection rate and exact classification accuracy.

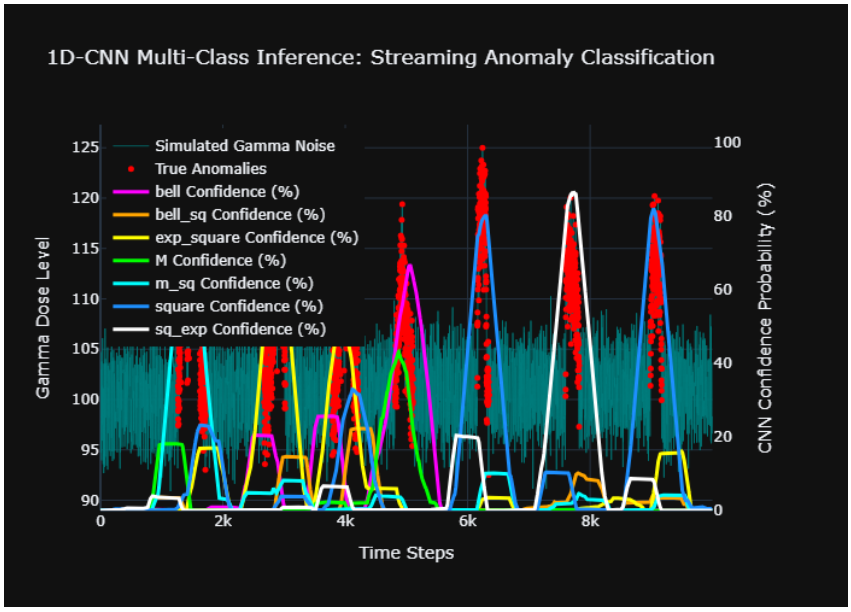

Figure 4: Multi-Class 1D-CNN Confidence Probability mapped against Simulated Gamma Noise